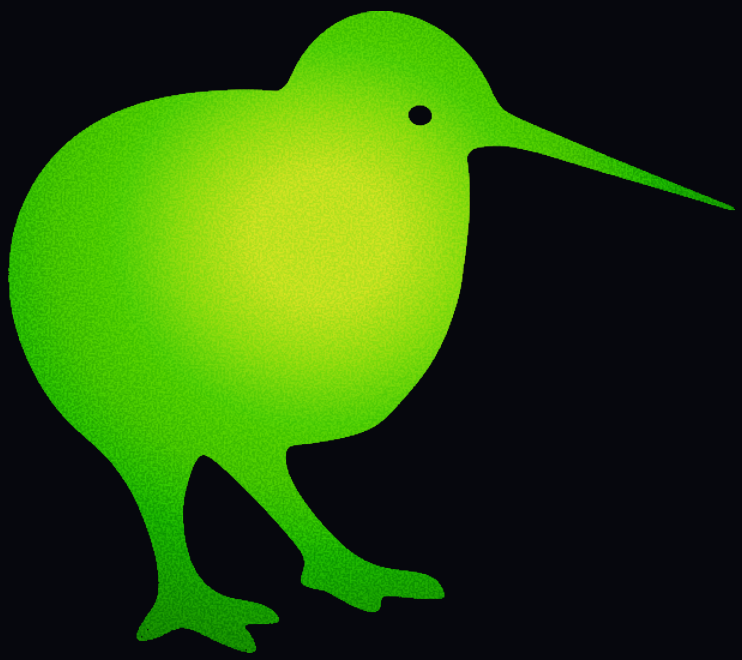
ROLE: BACKEND-FOCUSED FULL STACK

The Words Kivi Keeps

kivi
by Sarvam



Kivi can already turn speech into text across a person's computer. An ASR model hears the words; a language model turns unfinished speech into usable writing; Styles shape the result for different situations; shortcuts make the product quick to reach. Kivi also has a Dictionary.

It looks like a straightforward capability, but it took us a long time to understand what belonged inside it, what did not, and how something as small as a word could affect the entire experience.

The Dictionary emerged from a persistent problem. Even excellent speech recognition encounters language it cannot know in advance: the people in someone's life, the names inside their work, and the particular forms they expect to see when they speak.

We improved the models and the formatting, but one incorrect word could still make an otherwise excellent transcript feel impersonal. Over time, the question changed from "How do we fix this word?" to "What would it mean for Kivi to know this person?"

Kivi's larger ambition is to become a personal AI. The full vision for memory includes episodic memory, semantic memory, and factual knowledge about the person and their world.

This assignment concerns only its smallest useful edge: phonetic memory. Kivi should become better at recognising and producing the words that belong to its user.

The boundary is narrow. The problem inside it is not.

PART ONE

Build the System

Design and build Kivi's word-level memory system end to end.

Three versions of a transcript matter in this task:

1. **ASR output:** the raw text produced by the speech-recognition model;
2. **formatted output:** the text produced after Kivi's language model has cleaned and structured the speech;
3. **memory-aware output:** the text as it should appear once Kivi understands the relevant personal terms.

For example:

```
ASR output
ask aditya to review the sarvam kiwi service

Formatted output
Ask Aditya to review the Sarvam Kiwi service.

Memory-aware output
Ask Aaditya to review the Sarvam Kivi service.
```

This is the only example we will provide. It establishes the three transcript levels, not the boundaries of the problem. Discovering the other failures, ambiguities, and useful cases is part of the assignment.

Your job is to build the system that moves Kivi reliably from the first two versions to the third.

Begin with the product. Decide what it should mean for Kivi to remember a person's words, which observations deserve to change its future behaviour, and what the product should do when its evidence is weak or wrong.

Then make those decisions real. Determine how Kivi could learn through ordinary use, what it should store, how that knowledge should be kept durably, when it should be changed or removed, and how the relevant memory should be found and placed into the formatting prompt when the person speaks again.

We will not tell you where the memory-aware output comes from in the real product. We will not tell you how learning begins, how quickly it happens, what a memory entry contains, or how different memories relate to one another.

We will not provide Kivi's current implementation. These are product and engineering decisions, and they should follow from the kind of personal system you believe Kivi ought to be.

Do not build only for the example above. Show us the cases you discovered and what becomes possible because of your abstraction. Explain what your system can distinguish, why those distinctions matter, and where it should deliberately do nothing.

Build a runnable demonstration through which we can:

- provide the observations your system uses to learn;
- inspect the resulting memory state;
- provide new ASR and formatted outputs;
- see the memory-aware result;
- understand why the system did or did not intervene;
- reset the system and repeat the journey.

You choose what evidence the product learns from and how that evidence enters the system.

The demonstration may use a browser interface, desktop interface, command-line interface, or another coherent local application. It does not need to reproduce the production Kivi app.

You do not need to build speech recognition. You may use any language, database, model, or development tool.

PART TWO

Prove That It Works

How you evaluate the system is part of the assignment.

Define what success means. Create the data, cases, and expected behaviour required to test the product you chose to build.

The evaluation should go substantially beyond the example above. It should include situations in

which memory should affect the result and situations in which it should deliberately do nothing.

Make failures inspectable. For every case, preserve:

- the inputs;
- the expected result;
- the actual result;
- the relevant memory state;
- the reason the system made its decision.

Report useful interventions separately from unnecessary or incorrect interventions. Include latency, model usage, cost, and database growth wherever they matter to your approach.

Your evaluation must be reproducible. We should be able to run it ourselves, inspect individual cases, and verify the conclusions you present.

A collection of successful examples chosen after the system was built is not an evaluation.

Evaluations

We will not provide a hidden benchmark, private user corpus, or prescribed evaluation abstraction for this task.

We will run the evaluation included in your repository. We will inspect its data, methodology, expected outputs, actual outputs, and individual failures.

We will evaluate:

- whether the evaluation tests the actual product claim;
- whether it distinguishes useful learning from false or unnecessary intervention;
- whether its cases extend beyond the example in this brief;
- whether its conclusions are supported by reproducible evidence;
- whether memory state and model decisions are inspectable;
- whether latency, cost, and storage are measured honestly where relevant.

We will assess the system and the method used to evaluate it together. A strong-looking result supported by a weak evaluation will not be treated as strong work.

Submission

Place the complete submission in one GitHub repository.

The repository must contain:

- the complete source code;
- the runnable demonstration;
- the database schema and migrations;
- reproducible seed data;
- the complete evaluation and its dataset;
- generated evaluation results;
- a README explaining the product, architecture, decisions, limitations, and AI use;
- a `RUN.md` declaring the primary review method and explaining exactly how to use it.

Your application may be submitted as:

- a fully hosted application;
- a completely local application;
- a local interface connected to a hosted backend;
- a local interface and backend connected to a hosted database;
- a containerised application and database;
- another hybrid arrangement that the reviewing agent can reproduce.

Your database may be:

- SQLite, DuckDB, or another embedded database;
- Postgres, MySQL, or another locally running database;
- a database started through Docker;
- Supabase, Neon, Railway, or another managed database service;
- part of a fully hosted application.

All of these arrangements are valid. Deployment is not required.

Choose one arrangement as the primary review method. State it at the beginning of `RUN.md`.

For a hosted application, `RUN.md` must provide the direct URL, any required demonstration credentials, the interactions to try, and the evaluation procedure.

For a local or hybrid application, `RUN.md` must provide:

1. the required runtimes and versions;
2. every required environment variable;
3. the exact commands to install dependencies;
4. the exact commands to create, migrate, and seed the database;
5. the exact commands to start every required process;

6. the URL, application window, or interface to open;
7. the primary interactions to try;
8. the exact command to run the evaluation;
9. where the evaluation results are written;
10. the exact procedure for resetting the system.

If an LLM key is required, name the environment variable precisely and include an `.env.example`. Do not commit private credentials.

SUBMIT

Through the application form, provide:

- the GitHub repository URL;
- the exact final commit SHA;
- the hosted application URL, if the primary review method is hosted.

A coding agent will clone the submitted commit and follow the primary review method declared in `RUN.md`.

The agent may install declared dependencies and provide documented model credentials. It will not infer missing setup, perform undocumented dashboard work, repair the application, or contact you for clarification.

Test the complete review path from the submitted commit before applying.

If the agent cannot seamlessly start, operate, reset, and evaluate the system, we may be unable to review the submission.

Do not build the largest memory system you can describe.

Build the smallest one that makes Kivi feel as though it has met this person before.